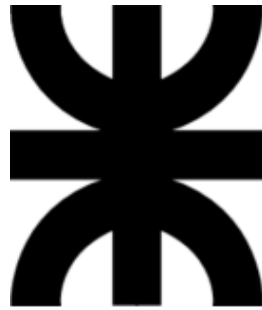


Universidad Tecnológica Nacional

Facultad Regional Tucumán



Trabajo Práctico Final

“Blog Personal”

Integrante

Iguzquiza, Leandro Agustín – 46349

Docente

Ing. Luis María Carriles

Cátedra

Virtualización de Servidores

Fecha de Presentación: 28/06/26

Comisión: 5K3

1. Introducción

El presente informe describe el proceso de desarrollo, configuración e implementación de un Blog Personal como Trabajo Práctico Final para la materia de Virtualización de Servidores, utilizando tecnologías libres y entornos virtualizados con contenedores Linux LXC en Proxmox VE. El sistema desarrollado permite visualizar los datos personales y una foto de perfil del alumno, y además administrar publicaciones dentro del blog mediante un formulario web integrado en la página principal.

2. Objetivo del proyecto

Implementar un servicio web funcional que cumpla con los siguientes requerimientos:

- Mostrar datos personales del alumno.
- Incluir una imagen personal como foto de perfil.
- Permitir agregar nuevas entradas al blog desde un formulario en la página principal.
- Publicar el presente informe en formato PDF dentro de la implementación.
- Ejecutar el servicio en contenedores LXC bajo un servidor Proxmox VE.
- Habilitar acceso público a través de nap.frt.utn.edu.ar.

3. Infraestructura y herramientas utilizadas

El entorno fue desplegado dentro de la infraestructura de la facultad, basada en Proxmox VE, con acceso remoto para estudiantes. Se utilizaron dos contenedores LXC con recursos de 128 MB de RAM, 1 procesador y 8 GB de almacenamiento cada uno, con direccionamiento IP estático en la red 172.16.90.0/24.

Contenedor 1 – Servidor Web (Blog Personal)

- **Nombre:** 41425819A
- **CT ID:** 189
- **IP:** 172.16.90.189/24
- **Gateway y DNS:** 172.16.90.1
- **Sistema operativo:** Ubuntu 22.04
- **Servicios:** Nginx, PHP-FPM 8.1, cliente MariaDB
- **Archivos implementados en /var/www/html/:**
 - **index.php:** interfaz principal con datos personales, formulario y listado de entradas
 - **config.php:** configuración de conexión PDO a la base de datos remota
 - **assets/foto.png:** imagen personal del alumno
 - **informe.pdf:** informe de desarrollo e implementación (publicado en el blog)

Contenedor 2 – Servidor de Base de Datos

- **Nombre:** 41425819DB
- **CT ID:** 190
- **IP:** 172.16.90.190/24
- **Gateway y DNS:** 172.16.90.1
- **Sistema operativo:** Ubuntu 22.04
 - **Servicio:** MariaDB Server
 - **Base de datos:** blog
 - **Tabla principal:** posts (id, titulo, contenido, fecha)

3.1 Justificación del stack tecnológico seleccionado

Para este trabajo se eligió un stack compuesto por Nginx, PHP-FPM y MariaDB, desplegado en dos contenedores LXC sobre Proxmox VE con Ubuntu 22.04.

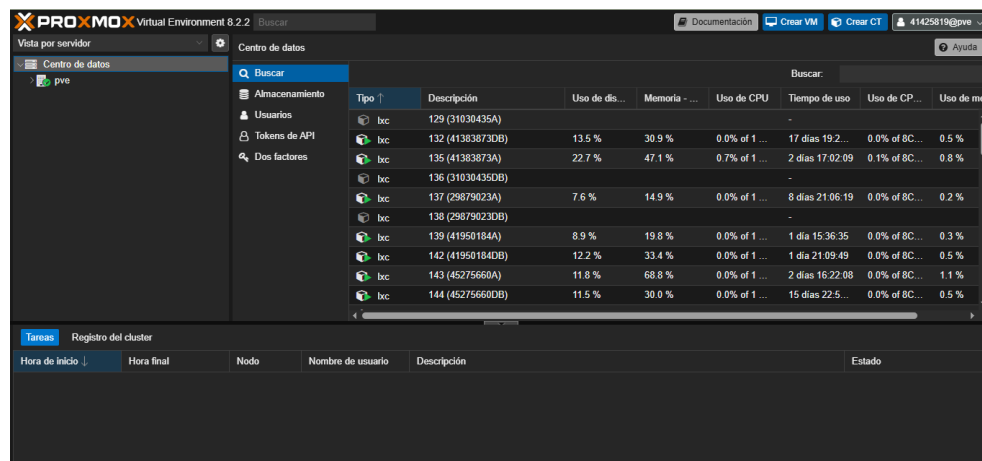
La consigna exige dos contenedores con recursos limitados (128 MB de RAM, 1 CPU y 8 GB de disco cada uno), separando la capa de aplicación web de la base de datos. Por este motivo se descartó concentrar todo en un único contenedor y se optó por una arquitectura cliente-servidor entre el blog y MariaDB.

Se eligió Nginx en lugar de Apache porque consume menos memoria y es adecuado para servir páginas PHP en entornos con pocos recursos. PHP-FPM permite procesar formularios y conectar con la base de datos de forma simple, cumpliendo el requisito de agregar entradas desde la página principal. MariaDB fue seleccionada por ser liviana, compatible con MySQL y apropiada para una base relacional pequeña como la del blog. PDO se utilizó para la conexión entre PHP y MariaDB porque facilita el manejo seguro de consultas parametrizadas y simplifica la conexión remota entre contenedores.

En conjunto, esta combinación permite cumplir con los requerimientos funcionales del TPF (datos personales, imagen, formulario de publicación e informe PDF), manteniendo un consumo de recursos acorde a las restricciones del entorno virtualizado provisto por la facultad.

4. Desarrollo del Trabajo

Para el desarrollo del presente trabajo, se empleó el entorno Proxmox VE con el fin de gestionar y optimizar los recursos virtuales.



ID	Descripción	Uso de dis...	Memoria - ...	Uso de CPU	Tiempo de uso	Uso de CP...	Uso de me
129 (31030435A)	lxc						
132 (41383873DB)	lxc	13.5 %	30.9 %	0.0% of 1 ...	17 días 19.2...	0.0% of 8C...	0.5 %
135 (41383873A)	lxc	22.7 %	47.1 %	0.7% of 1 ...	2 días 17:02:09	0.1% of 8C...	0.8 %
136 (31030435DB)	lxc						
137 (29879023A)	lxc	7.6 %	14.9 %	0.0% of 1 ...	8 días 21:06:19	0.0% of 8C...	0.2 %
138 (29879023DB)	lxc						
139 (41950184A)	lxc	8.9 %	19.8 %	0.0% of 1 ...	1 día 15:36:35	0.0% of 8C...	0.3 %
142 (41950184DB)	lxc	12.2 %	33.4 %	0.0% of 1 ...	1 día 21:09:49	0.0% of 8C...	0.5 %
143 (45275660A)	lxc	11.8 %	68.8 %	0.0% of 1 ...	2 días 16:22:08	0.0% of 8C...	1.1 %
144 (45275660DB)	lxc	11.5 %	30.0 %	0.0% of 1 ...	15 días 22.5...	0.0% of 8C...	0.5 %

Figura 1: Acceso al entorno Proxmox VE

4.1. Creación de los contenedores Linux LXC

Se crearon dos contenedores LXC en el nodo pve de Proxmox VE, siguiendo la nomenclatura establecida por la cátedra (DNI + identificador). La IP de cada contenedor se construyó a partir del CT ID asignado por Proxmox, utilizando el formato 172.16.90.{ID} con máscara /24 (255.255.255.0), gateway 172.16.90.1 y DNS 172.16.90.1.



Figura 2: Asistente de creación de contenedor LXC

Crear: Contenedor LXC

General Plantilla Discos CPU Memoria Red DNS Confirmar

Key ↑	Value
cores	1
features	nesting=1
hostname	41425819A
memory	128
nameserver	172.16.90.1
net0	name=eth0,bridge=vmbr0,firewall=1,ip=172.16.90.189/24,gw=172.16.90.1
nodename	pve
ostemplate	local:vztmpl/ubuntu-22.04-standard_22.04-1_amd64.tar.zst
pool	
rootfs	local-lvm:8
ssh-public-keys	
swap	128
unprivileged	1
vmid	189

☐ Iniciar después de la creación

Avanzado ☐ Atrás Finalizar

Figura 3: Configuración general del contenedor 41425819A (CT 189)

General Plantilla Discos CPU Memoria Red DNS Confirmar

Key ↑	Value
cores	1
features	nesting=1
hostname	41425819DB
memory	128
nameserver	172.16.90.1
net0	name=eth0,bridge=vmbr0,firewall=1,ip=172.16.90.190/24,gw=172.16.90.1
nodename	pve
ostemplate	local:vztmpl/ubuntu-22.04-standard_22.04-1_amd64.tar.zst
pool	
rootfs	local-lvm:8
ssh-public-keys	
swap	128
unprivileged	1
vmid	190

☐ Iniciar después de la creación

Figura 4: Configuración general del contenedor 41425819DB (CT 190)

4.2. Instalación de herramientas necesarias

Contenedor de base de datos (172.16.90.190)

Se instaló MariaDB Server y se configuró para aceptar conexiones remotas:

```
apt update
apt install -y mariadb-server
```

Contenedor web (172.16.90.189)

En el contenedor del blog se instaló **Nginx** como servidor web y **PHP-FPM** como motor de procesamiento de scripts PHP, junto con la extensión necesaria para conectarse a **MariaDB**, mediante los siguientes comandos:

```
apt update
apt install -y nginx php-fpm php-mysql
systemctl enable nginx php8.1-fpm
systemctl start nginx php8.1-fpm
```

La configuración de **Nginx** se realizó mediante un sitio virtual dedicado, apuntando al directorio `/var/www/html/`, donde se alojan los archivos del blog. Se habilitó el procesamiento de archivos PHP y se desactivó el sitio por defecto de **Nginx** para evitar conflictos.

Posteriormente, mediante `curl -I http://127.0.0.1/`, se verificó que **Nginx** respondiera correctamente en el puerto 80 del contenedor, lo que confirmó que la capa web estaba lista para alojar la aplicación.

4.3. Creación del usuario, base de datos y tabla

Con MariaDB operativo, se creó la base de datos blog, destinada a almacenar las publicaciones del sitio.

Para permitir la comunicación entre contenedores, se creó el usuario “bloguser”, restringido específicamente a conectarse desde la IP del contenedor web (172.16.90.189). De esta forma, la base de datos no queda accesible de forma general, sino únicamente desde el servicio autorizado.

Sobre la base blog se creó la tabla posts, con campos para identificador, título, contenido y fecha de publicación. Esta estructura resulta suficiente para cumplir con el requerimiento de permitir agregar nuevas entradas al blog desde la página principal.

Posteriormente, se verificó la conexión desde el contenedor web mediante el cliente MariaDB, confirmando que la comunicación entre ambos contenedores funcionaba correctamente.

Desde el contenedor de base de datos se accedió a MariaDB y se ejecutó:

```
CREATE DATABASE blog CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

CREATE USER 'bloguser'@'172.16.90.189' IDENTIFIED BY
'TU_PASSWORD_SEGURA';
GRANT SELECT, INSERT, UPDATE, DELETE ON blog.* TO
'bloguser'@'172.16.90.189';
FLUSH PRIVILEGES;
```

```
USE blog;

CREATE TABLE posts (
  id INT AUTO_INCREMENT PRIMARY KEY,
  titulo VARCHAR(200) NOT NULL,
  contenido TEXT NOT NULL,
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

El usuario “bloguser” solo puede conectarse desde la IP del contenedor web (172.16.90.189), lo que limita el acceso a la base de datos únicamente al servicio del blog.

```
MariaDB [(none)]> CREATE DATABASE blog CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
REATE USER 'bloguser'@'172.16.90.189' IDENTIFIED BY 'TU_PASSWORD_SEGURA';
GRANT SELECT, INSERT, UPDATE, DELETE ON blog.* TO 'bloguser'@'172.16.90.189';
FLUSH PRIVILEGES;

USE blog;

CREATE TABLE posts (
  id INT AUTO_INCREMENT PRIMARY KEY,
  titulo VARCHAR(200) NOT NULL,
  contenido TEXT NOT NULL,
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);Query OK, 1 row affected (0.339 sec)

MariaDB [(none)]>
MariaDB [(none)]> CREATE USER 'bloguser'@'172.16.90.189' IDENTIFIED BY 'TU_PASSWORD_SEGURA';
Query OK, 0 rows affected (0.350 sec)

MariaDB [(none)]> GRANT SELECT, INSERT, UPDATE, DELETE ON blog.* TO 'bloguser'@'172.16.90.189';
Query OK, 0 rows affected (0.392 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.055 sec)

MariaDB [(none)]>
MariaDB [(none)]> USE blog;
Database changed
MariaDB [blog]>
MariaDB [blog]> CREATE TABLE posts (
  -> id INT AUTO_INCREMENT PRIMARY KEY,
  -> titulo VARCHAR(200) NOT NULL,
  -> contenido TEXT NOT NULL,
  -> fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  -> );
```

Figura 5: Creación de base de datos, usuario y tabla posts

4.4. Creación de config.php e index.php

La aplicación del blog se desarrolló en PHP y se desplegó en el directorio /var/www/html/ del contenedor web.

Se creó el archivo config.php, responsable de establecer la conexión con la base de datos remota utilizando PDO. En este archivo se definieron la IP del contenedor de base de datos, el nombre de la base, el usuario autorizado y la contraseña correspondiente.

El archivo index.php constituye la interfaz principal del blog e integra todos los requerimientos del trabajo:

- una sección con datos personales del alumno;
- la imagen de perfil;
- un formulario para cargar nuevas entradas;
- un listado de publicaciones ya almacenadas;

- un enlace al informe PDF del TPF.

Cuando el usuario completa el formulario y envía una entrada, la aplicación inserta los datos en la tabla posts de MariaDB y luego vuelve a mostrar la página actualizada.

```
<?php
$db_host = '172.16.90.190';
$db_name = 'LeoIguz';
$db_user = 'LeandroIguzquiza';
$db_pass = '41425819';

try {
    $pdo = new PDO(
        "mysql:host=$db_host;dbname=$db_name;charset=utf8mb4",
        $db_user,
        $db_pass,
        [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
    );
} catch (PDOException $e) {
    die('Error de conexión a la base de datos.');
```

Figura 6: Archivo config.php con credenciales de conexión

index.php — interfaz principal del blog. Incluye: sección de datos personales del alumno (nombre, carrera, email, DNI), imagen de perfil, formulario para agregar entradas (título y contenido), listado de entradas ordenadas por fecha descendente, y enlace al informe del Trabajo Practico Final.

```
GNU nano 6.2 index.php
<div>
  <h1>Mi Blog Personal</h1>
  <p><strong>Nombre:</strong>Leandro Agustin Iguzquiza</p>
  <p><strong>Carrera:</strong> Ingenieria en Sistemas de Informacion</p>
  <p><strong>Email:</strong> leandroiguz@gmail.com</p>
  <p><strong>DNI:</strong> 41425819</p>
</div>
</section>

<h2>Nueva entrada</h2>
<form method="POST">
  <input type="text" name="titulo" placeholder="T tulo" required>
  <textarea name="contenido" rows="4" placeholder="Contenido de la entrada" required></textarea>
  <button type="submit">Publicar</button>
</form>

<h2>Entradas del blog</h2>
<?php if (empty($posts)): ?>
  <p>No hay entradas todavia.</p>
<?php else: ?>
  <?php foreach ($posts as $post): ?>
    <article class="post">
      <h3><?= htmlspecialchars($post['titulo']) ?></h3>
      <small><?= htmlspecialchars($post['fecha']) ?></small>
      <p><?= nl2br(htmlspecialchars($post['contenido'])) ?></p>
    </article>
  </foreach>
</else>
</if>
</h2>
```

Figura 7: Archivo index.php, interfaz principal del blog personal

4.5. Imagen personal y publicación del informe

La imagen personal del alumno se almacenó en la carpeta `/var/www/html/assets/`, desde donde Nginx puede servirla como recurso estático.

Inicialmente se evaluaron distintas formas de transferir la imagen al contenedor, considerando limitaciones de acceso remoto y de consola. Finalmente, la imagen fue obtenida desde un repositorio público en GitHub mediante descarga directa al contenedor, y luego se ajustaron permisos para que el servidor web pudiera leerla correctamente.

Debido a que el acceso público al sitio se realiza a través del NAP con la ruta `/41425819/`, fue necesario adaptar la estructura de archivos para que los recursos estáticos también fueran accesibles bajo ese prefijo. Para ello se crearon enlaces simbólicos que permiten servir tanto la aplicación como la carpeta de assets desde la ruta pública requerida por la facultad.

```
mkdir -p /var/www/html/41425819
ln -sf /var/www/html/assets /var/www/html/41425819/assets
ln -sf /var/www/html/index.php /var/www/html/41425819/index.php
```

En el HTML se utilizó la ruta absoluta `/41425819/assets/foto.png` para la imagen de perfil.

4.6. Configuración de firewall en Proxmox

Durante la creación del contenedor web, se habilitó el firewall del CT en Proxmox. Para permitir el acceso HTTP desde el NAP hacia el servicio del blog, se configuró una regla de entrada que acepta tráfico **TCP al puerto 80**.

Esta regla resulta necesaria para que las peticiones externas puedan llegar correctamente al servidor Nginx alojado en el contenedor.

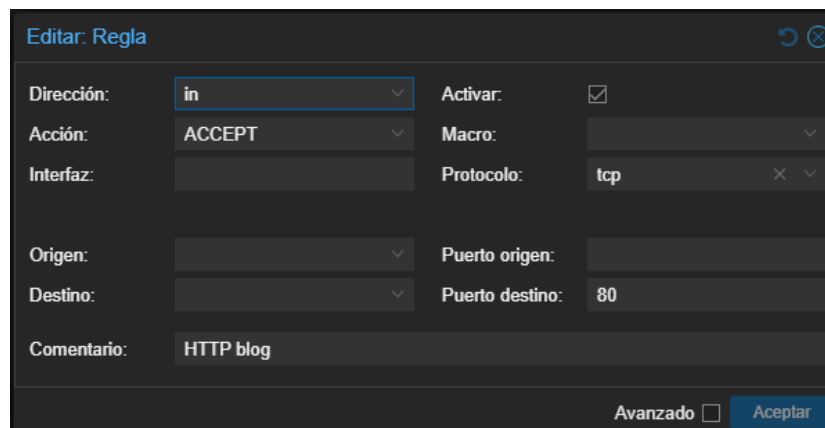


Figura 8: Regla de firewall del contenedor 41425819A para permitir tráfico HTTP entrante (TCP puerto 80)

4.7. Pruebas de funcionamiento

Durante la etapa de pruebas se verificaron distintos aspectos del sistema:

- funcionamiento del servidor web en el contenedor;
- conexión entre PHP y MariaDB;

- carga de entradas mediante formulario;
- persistencia de datos en la base;
- disponibilidad del enlace al informe PDF.

Las pruebas locales se realizaron mediante solicitudes HTTP al propio contenedor, confirmando que la aplicación respondía correctamente. También se verificó, desde el contenedor web, que los datos insertados quedaran registrados en la tabla posts de MariaDB.

Durante el desarrollo, y ante dificultades temporales para acceder al sitio desde redes externas, se utilizó **Cloudflare Tunnel** como herramienta auxiliar para visualizar el blog mientras se gestionaba el acceso definitivo por NAP. Esta alternativa permitió validar el funcionamiento general de la aplicación, aunque la publicación final del trabajo se realiza a través de la infraestructura de la facultad.

```
root@41425819DB:~# mysql -u root -p -e "USE blog; SELECT * FROM posts;"
Enter password:
+----+-----+-----+
| id | titulo          | contenido |
+----+-----+-----+
|    |    | fecha          |
+----+-----+-----+
| 1 | Entrada de Prueba | Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. | 2026-06-14 00:17:18 |
+----+-----+-----+
```

Figura 9: Verificación del servicio MariaDB activo en el contenedor 41425819DB

```
root@41425819A:/# systemctl status nginx
* nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2026-06-16 18:10:52 UTC; 1 week 0 days ago
     Docs: man:nginx(8)
  Process: 101 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
  Process: 293 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
 Main PID: 294 (nginx)
    Tasks: 2 (limit: 9332)
  Memory: 2.5M
     CPU: 37ms
  CGroup: /system.slice/nginx.service
          └─294 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
            └─295 "nginx: worker process"

Jun 16 18:10:47 41425819A systemd[1]: Starting A high performance web server and a reverse proxy server...
Jun 16 18:10:52 41425819A systemd[1]: Started A high performance web server and a reverse proxy server.
lines 1-16/16 (END)
```

Figura 10: Verificación del servicio Nginx activo en el contenedor web 41425819A

[illegible]

Figura 11: Verificación del servicio PHP-FPM activo en el contenedor web 41425819A

```

root@41425819A:/# curl -I http://127.0.0.1/
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Tue, 23 Jun 2026 23:01:22 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive

```

Figura 12: Prueba de respuesta HTTP del blog en localhost (200 OK)

```

root@41425819DB:~# systemctl status mariadb
lines 1--1 ...skipping...
* mariadb.service - MariaDB 10.6.23 database server
   Loaded: loaded (/lib/systemd/system/mariadb.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-06-13 16:33:00 UTC; 1 week 3 days ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 1527 ExecStartPre=/usr/bin/install -m 755 -o mysql -g root -d /var/run/mysqld (code=exited, status=0/SUCCESS)
   Process: 1528 ExecStartPre=/bin/sh -c systemctl unset-environment _WSREP_START_POSITION (code=exited, status=0/SUCCESS)
   Process: 1530 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] && VAR= || VAR=/usr/bin/galera_recovery; (code=exited, status=0/SUCCESS)
   Process: 1581 ExecStartPost=/bin/sh -c systemctl unset-environment _WSREP_START_POSITION (code=exited, status=0/SUCCESS)
   Process: 1583 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUCCESS)
  Main PID: 1568 (mariabdd)
    Status: "Taking your SQL requests now..."
     Tasks: 9 (limit: 61592)
    Memory: 12.4M
       CPU: 40.547s
   CGroup: /system.slice/mariadb.service
           └─1568 /usr/sbin/mariabdd

Jun 13 16:33:00 41425819DB /etc/mysql/debian-start[1588]: Looking for 'mariadb' as: /usr/bin/mariadb
Jun 13 16:33:00 41425819DB /etc/mysql/debian-start[1588]: Looking for 'mariadb-check' as: /usr/bin/mariadb-check
Jun 13 16:33:00 41425819DB /etc/mysql/debian-start[1588]: This installation of MariaDB is already upgraded to 10.6.23-MariaDB
Jun 13 16:33:00 41425819DB /etc/mysql/debian-start[1588]: There is no need to run mysql upgrade again.
Jun 13 16:33:00 41425819DB /etc/mysql/debian-start[1588]: You can use --force if you still want to run mysql upgrade
Jun 13 16:33:00 41425819DB /etc/mysql/debian-start[1596]: Checking for insecure root accounts.
Jun 13 16:33:00 41425819DB /etc/mysql/debian-start[1600]: Triggering myisam-recover for all MyISAM tables and aria-recover
Jun 13 16:35:21 41425819DB mariabdd[1568]: 2026-06-13 16:35:21 31 [Warning] IP address '172.16.90.189' could not be resolved
Jun 13 16:35:21 41425819DB mariabdd[1568]: 2026-06-13 16:35:21 31 [Warning] Access denied for user 'bloguser'@'172.16.90.189'

```

Figura 13: Consulta de entradas registradas en la tabla posts de MariaDB

4.8. Acceso público mediante NAP

El acceso externo al blog se realiza a través de:

<http://nap.frt.utn.edu.ar/41425819/>

El contenedor web escucha internamente en el **puerto 80**, mientras que el NAP expone el servicio mediante la ruta asociada al DNI del alumno. De esta forma, el blog queda accesible desde Internet cumpliendo con lo establecido en la consigna del trabajo práctico.

Para habilitar este acceso, se comunicó al docente la IP del contenedor web (**172.16.90.189**), el puerto interno del servicio (**80**) y el identificador correspondiente al DNI (**41425819**), con el fin de que el NAP pudiera enrutar correctamente las peticiones hacia la implementación.



Figura 14: Vista del blog personal accesible desde nap.frt.utn.edu.ar/41425819/

5. Conclusión

El Trabajo Práctico Final fue implementado cumpliendo con los requisitos solicitados por la cátedra: despliegue sobre contenedores LXC, separación entre servicio web y base de datos, visualización de datos personales, incorporación de imagen del alumno, posibilidad de agregar entradas desde la página principal y publicación del informe dentro del sitio.

La experiencia permitió aplicar conceptos de virtualización, configuración de red estática, administración de servicios en Linux, conectividad entre contenedores y desarrollo web con PHP y MariaDB en un entorno con recursos limitados.

Entre las dificultades encontradas se destacan las restricciones de memoria de los contenedores, la configuración de acceso remoto a MariaDB, la adaptación de rutas estáticas al prefijo del NAP y la necesidad de coordinar con el docente la habilitación del acceso público. No obstante, la solución implementada resultó adecuada para cumplir con los objetivos planteados y demostrar la consolidación de servicios en un entorno virtualizado.